

MACs y Cifrado Autenticado

Guía de Estudio — Criptografía y Seguridad (72.44) — ITBA

Unidad 3: Integridad, CBC-MAC, HMAC, Cifrado Autenticado

Índice

1. ¿Qué es un MAC?	1
2. Experimento Mac-Forge	2
3. CBC-MAC (longitud fija)	3
3.1. Ataque de extensión (longitud variable)	3
3.2. Extensiones seguras para longitud variable	3
4. Funciones de Hash	4
5. HMAC	5
6. Cifrado Autenticado: las 3 construcciones	6
6.1. Definición formal de Encrypt-then-MAC	6
6.2. Modos AEAD en la práctica	6
7. No-repudio: por qué los MACs no lo dan	7
8. Memory hooks	7
9. Ejercicios tipo examen con respuesta	7
9.1. V/F: hash, MAC y cifrado (parciales 2016/2018/2023/2025)	7
9.2. Encrypt-and-MAC inseguro (parciales 2016/2018/2023)	8
9.3. HMAC para CVV (parcial 2017)	8
9.4. Construcción $c = E_{k_1}(m \parallel H(k_2 \parallel m))$ (parcial 2025-1)	8

1. ¿Qué es un MAC?

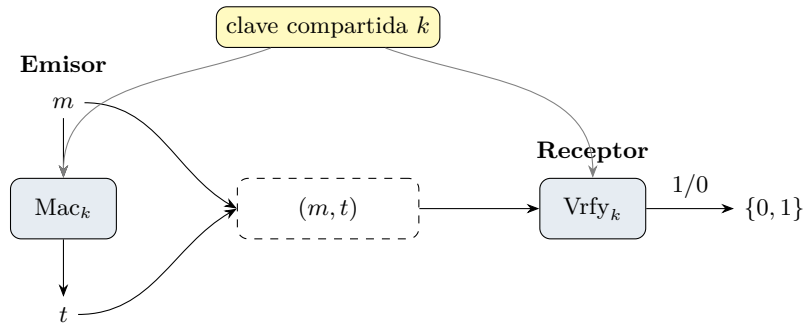
MAC (Message Authentication Code)

Terna de algoritmos (Gen, Mac, Vrfy):

- $\text{Gen} : () \rightarrow K$ — generador de clave.
- $\text{Mac} : K \times \mathcal{M} \rightarrow \mathcal{T}$ — etiquetador (genera la *etiqueta*/tag t).
- $\text{Vrfy} : K \times \mathcal{M} \times \mathcal{T} \rightarrow \{0, 1\}$ — verificador.

Correctitud: para todo m y clave válida k : $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$.

Objetivo: integridad sin (necesariamente) confidencialidad. El MAC *no* cifra el mensaje; protege contra modificaciones no autorizadas. A diferencia de un hash criptográfico, **el MAC tiene clave**: sin la clave k nadie puede generar un tag válido. A diferencia de un criptosistema, no hay cifrado/descifrado, solo etiquetado y verificación.



Emisor calcula $t = \text{Mac}_k(m)$ y envía (m, t) ; receptor corre $\text{Vrfy}_k(m, t)$ y obtiene 1 (íntegro) o 0 (alterado). La misma clave k se comparte entre ambos.

Destacado

Diferencia clave MAC vs. hash: un hash es público (sin clave); cualquiera puede calcularlo. Un MAC requiere la clave k : sin ella, no se puede generar ni verificar una etiqueta válida. Por eso el MAC da *autenticación de origen* y no solo integridad.

2. Experimento Mac-Forge

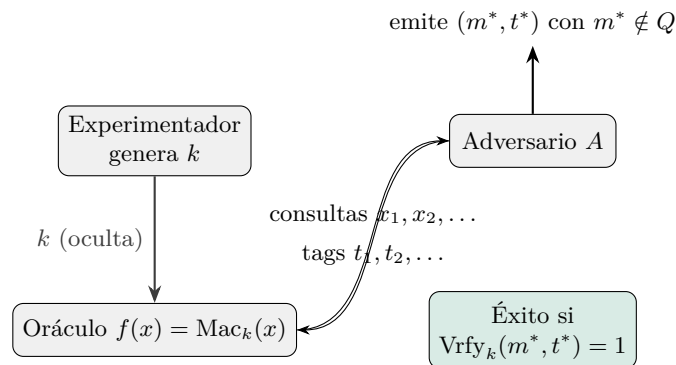
Experimento $\text{Mac-Forge}_{A, \Pi}(n)$

Dado nivel de seguridad n , adversario A y MAC Π :

1. Se genera $k \leftarrow K$.
2. A recibe acceso a un **oráculo de etiquetado** $f(x) = \text{Mac}_k(x)$ (caja negra; nunca ve k).
3. A realiza todas las consultas que desee. Sea Q el conjunto de mensajes consultados.
4. A emite (m^*, t^*) con $m^* \notin Q$.

El experimento vale 1 (*falsificación*) si $\text{Vrfy}_k(m^*, t^*) = 1$.

Π es **infalsificable** (EUF-CMA) si $\Pr[\text{Mac-Forge}_{A, \Pi}(n) = 1] \leq \text{neg}(n)$ para todo PPT A .



Protocolo del experimento Mac-Forge. El adversario puede consultar el oráculo libremente, pero su falsificación debe ser sobre un mensaje nuevo.

¿Por qué necesita el oráculo? En un ataque real el adversario ve muchos pares (m_i, t_i) en tránsito (mensajes legítimos). El oráculo modela eso. Sin oráculo, la definición sería demasiado débil: bastaría resistir solo el caso en que el atacante no observa ningún par.

Diferencia con CPA. En CPA el adversario quiere distinguir entre dos cifrados; en Mac-Forge quiere *crear* un par válido nuevo. CPA trabaja sobre confidencialidad; Mac-Forge trabaja sobre integridad. Además, en CPA el oponente elige los dos mensajes candidatos y luego recibe uno; en Mac-Forge puede consultar cualquier mensaje antes de emitir la falsificación.

¡OJO!

La restricción $m^* \notin Q$ evita el ataque trivial de reusar un tag ya obtenido. Una falsificación válida exige generar un tag *para un mensaje nunca consultado*, sin conocer k .

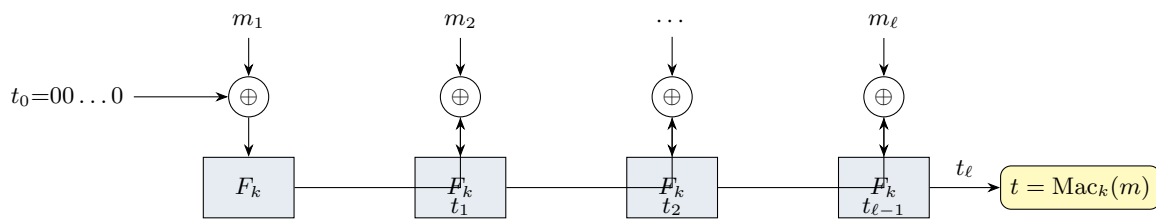
3. CBC-MAC (longitud fija)

CBC-MAC (longitud de mensaje fija)

Sea F_k una función pseudoaleatoria (PRF). Para mensaje $m = m_1 \| m_2 \| \dots \| m_\ell$:

$$t_0 = 0 \dots 0, \quad t_i = F_k(t_{i-1} \oplus m_i), \quad \text{Mac}_k(m) = t_\ell.$$

Verificación: recalcular y comparar (determinista).



CBC-MAC: IV fijo en cero, encadenamiento $t_i = F_k(t_{i-1} \oplus m_i)$, se descartan los intermedios y se emite solo t_ℓ .

¿Por qué IV = 0? En cifrado CBC el IV debe ser aleatorio para que el cifrado sea no determinista (necesario para CPA-Security). En CBC-MAC *no nos interesa* no determinismo: queremos que la misma clave y el mismo mensaje siempre den el mismo tag (necesario para la verificación). Por eso el IV es constante (cero).

Seguro solo para longitud fija. La prueba de seguridad de CBC-MAC asume que todos los mensajes tienen exactamente ℓ bloques. Si se permiten mensajes de distinta longitud, aparecen ataques de extensión.

3.1. Ataque de extensión (longitud variable)

[Examen]

Construcción de la falsificación de 3 bloques (parcial 2016):

1. Consultar $t_A = f(A)$ (MAC del bloque único A).
2. Consultar $t_{AB} = f(A \| B)$ (MAC del mensaje de 2 bloques $A \| B$).
3. **Emitir** el mensaje de 3 bloques $m^* = A \| B \| (A \oplus t_A)$ con tag $t^* = t_{AB}$.

Por qué valida: al procesar m^* , el estado después del segundo bloque es t_A (igual que el MAC de A consultado). El tercer bloque entra como $F_k(t_A \oplus (A \oplus t_A)) = F_k(A) = t_A$. Luego el estado final es $F_k(t_A) = t_{AB}$ (el MAC de $A \| B$ ya conocido). Falsificación válida sin conocer k , con $m^* \notin Q$.

3.2. Extensiones seguras para longitud variable

1. **Clave dependiente de la longitud.** Derivar $k' = F_k(|m|)$ y usar k' para CBC-MAC. Mensajes de distinta longitud usan claves distintas; no se aprenden estados intermedios. Requiere conocer $|m|$ antes de comenzar.

2. **Longitud como prefijo.** $\text{Mac}_k(m) = \text{CBC-MAC}_k(|m| \parallel m)$. Un mensaje no puede ser prefijo de otro (distinta longitud, distinto primer bloque). También requiere conocer $|m|$ a priori.
3. **Doble clave (EMAC).** Calcular $t' = \text{CBC-MAC}_{k_1}(m)$ y luego $t = F_{k_2}(t')$. No requiere conocer $|m|$ de antemano; usa el doble de material de clave.

¡OJO!

Agregar $|m|$ como **sufijo** (al final del mensaje) **NO es seguro**: el atacante ya procesó todos los bloques antes de llegar a la longitud, y puede explotar eso con un ataque similar al de extensión.

4. Funciones de Hash

Función de hash criptográfica

Par de algoritmos (S, H) : S selecciona una función de una familia (parámetro **público**, sin clave); H convierte un mensaje de longitud arbitraria en una etiqueta (digest) de tamaño **fijo y reducido**. **No tiene clave** — diferencia fundamental con MAC y criptosistemas.

Tres propiedades de resistencia

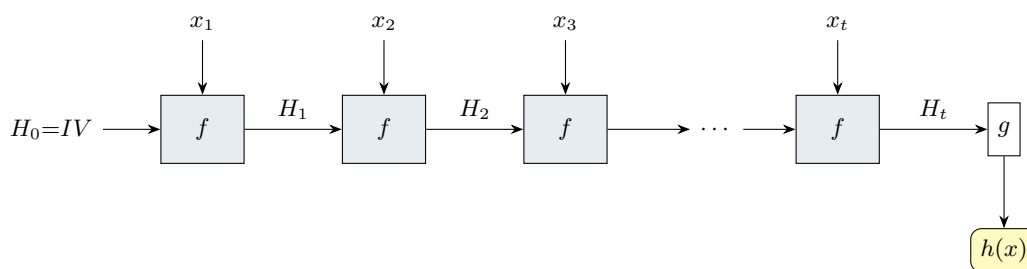
Sea b la cantidad de bits de salida.

- **Resistencia a preimagen.** Dado y , imposible hallar x con $h(x) = y$. Costo fuerza bruta: $\sim 2^b$.
- **Resistencia a segunda preimagen.** Dado x , imposible hallar $x' \neq x$ con $h(x') = h(x)$. Costo: $\sim 2^b$.
- **Resistencia a colisión.** Imposible hallar cualquier par $x \neq x'$ con $h(x) = h(x')$. Costo: $\sim 2^{b/2}$ (*paradoja del cumpleaños*). Es la más fuerte: implica las otras dos.

Paradoja del cumpleaños. Al buscar una colisión se compara cada hash nuevo contra *todos* los anteriores, no contra un objetivo fijo. La probabilidad de colisión crece como la raíz cuadrada del espacio; con b bits de salida basta generar $\sim 2^{b/2}$ mensajes. Por eso el nivel de seguridad efectivo frente a colisiones es **la mitad de los bits de salida**.

Propiedad	Objetivo	Costo FB
Preimagen	dado y , hallar x : $h(x) = y$	$\sim 2^b$
2. ^a preimagen	dado x , hallar $x' \neq x$ con igual hash	$\sim 2^b$
Colisión	hallar cualquier par $x \neq x'$ colisionante	$\sim 2^{b/2}$

Construcción de Merkle-Damgård. La mayoría de las funciones clásicas (MD5, SHA-1, SHA-2) siguen este esquema: preprocesamiento (padding + bloque con la longitud) seguido de iteración de una función de compresión f encadenando estados desde $H_0 = IV$.



Merkle-Damgård: mensaje $x = x_1 \cdots x_t$ procesado bloque a bloque por f ; transformación final g produce $h(x) = g(H_t)$.

Funciones en la práctica.

- **MD5** (128 bits): *rota*, no usar.
- **SHA-1** (160 bits): borderline, complicada.
- **SHA-2** (224/256/384/512 bits): estándar durante años; ~ 1 orden de magnitud más lenta que SHA-1.
- **SHA-3/Keccak**: ganador del concurso abierto NIST; mismos tamaños que SHA-2, velocidad de SHA-1, estructura completamente distinta. **Estándar recomendado hoy**; 256 bits alcanza y sobra.

¡OJO!

Errores típicos de V/F (parciales 2025):

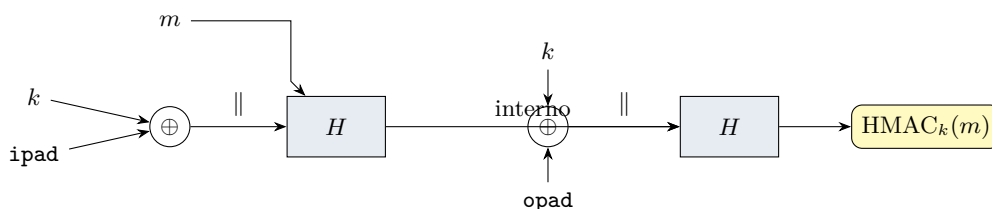
- “*Un hash sirve para encriptar*” $\rightarrow$ **FALSO**. El hash no es reversible; no permite recuperar el mensaje.
- “*MD5 es un criptosistema asimétrico*” $\rightarrow$ **FALSO**. MD5 es una función de hash, sin clave y sin descifrado.

5. HMAC

HMAC (RFC 2104)

Dada una función de hash H , clave k y constantes públicas ipad , opad :

$$\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m)).$$



HMAC: hash interno $H((k \oplus \text{ipad}) \parallel m)$ encapsulado dentro del hash externo $H((k \oplus \text{opad}) \parallel (\cdot))$. El mensaje se procesa una sola vez, de ahí la eficiencia.

Por qué es más rápido que CBC-MAC. Las funciones de hash están diseñadas para procesar grandes volúmenes de datos muy rápido (operaciones simples de bit). CBC-MAC requiere una evaluación de cifrado de bloque por bloque de mensaje; HMAC solo dos pasadas de hash (y el hash externo solo toca la etiqueta interna, no el mensaje completo).

Demostración formal. Si H es resistente a colisiones, HMAC es infalsificable. En sistemas reales: HMAC-SHA256, HMAC-SHA3-256. Evitar HMAC-MD5 y HMAC-SHA1 en diseños nuevos.

[Examen]

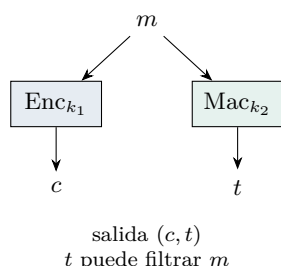
Validar sin exponer el secreto (parcial 2017, caso CVV). Se quiere verificar un dato sensible (ej. el CVV de una tarjeta) sin almacenarlo ni transmitirlo en claro: se almacena/compara un HMAC con clave simétrica. Quien no tiene la clave no puede generar ni validar el tag.

6. Cifrado Autenticado: las 3 construcciones

Construcción	Idea	¿Seguro?
Encrypt-and-MAC (paralelo)	Cifrar m y etiquetar m en paralelo; salida (c, t)	NO
MAC-then-Encrypt	Etiquetar m , luego cifrar $(m t)$; salida c	<i>Puede</i>
Encrypt-then-MAC	Cifrar m obteniendo c ; luego etiquetar c ; salida (c, t)	SÍ

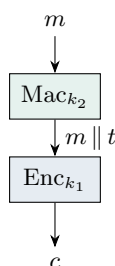
Encrypt-and-MAC

INSEGURO



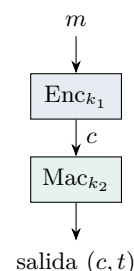
MAC-then-Encrypt

puede ser seguro



Encrypt-then-MAC

SÍ (cifrado autenticado)



MAC sobre el cifrado c
claves $k_1 \neq k_2$

Las tres construcciones. **Encrypt-and-MAC**: inseguro; el tag de m puede filtrar información.

MAC-then-Encrypt: sin prueba genérica. **Encrypt-then-MAC**: siempre seguro; es el “cifrado autenticado”.

6.1. Definición formal de Encrypt-then-MAC

Cifrado autenticado (Encrypt-then-MAC)

Requiere: un criptosistema **CPA-Secure** y un MAC **infalsificable**, con **claves independientes** $k_1 \neq k_2$.

- Gen: genera (k_1, k_2) independientemente.
- Cifrado: $c \leftarrow \text{Enc}_{k_1}(m)$; salida $(c, t = \text{Mac}_{k_2}(c))$.
- Descifrado: **primero** verificar $\text{Vrfy}_{k_2}(c, t)$; si falla, rechazar (error/excepción); si pasa, devolver $\text{Dec}_{k_1}(c)$.

Destacado

Clave: MAC sobre el **CIFRADO** c , con **claves independientes** $k_1 \neq k_2$. Aparece en parciales 2016, 2018, 2023. Si alguien manipula el texto cifrado, la verificación **falla** (antes de descifrar), eliminando los ataques de maleabilidad.

6.2. Modos AEAD en la práctica

- **CCM** (Counter with CBC-MAC): variante MAC-then-Encrypt (prueba específica). Calcula el CBC-MAC y lo cifra junto al mensaje en modo contador. Una sola clave; dos pasadas lógicas por el mensaje. Seguro si IV y nonce no se repiten.
- **GCM** (Galois Counter Mode): cifrado en modo contador + GHASH (aritmética de polinomios sobre $\text{GF}(2^{128})$). Mucho más rápido que CCM. **AES-GCM** es la recomendación práctica: rápido, estandarizado, libre de patentes, en todas las librerías serias.

7. No-repudio: por qué los MACs no lo dan

¡OJO!

ERROR TÍPICO MUY IMPORTANTE (parciales 2025-1 Ej.3d, 2025-2 Ej.5b): afirmar que un MAC/HMAC da **no-repudio**.

NO lo da. La clave simétrica es *compartida*: ambas partes (emisor y receptor) tienen la misma k y *ambas* pueden generar un tag válido. Por lo tanto, el receptor no puede probar ante un tercero que el mensaje lo originó el emisor (él mismo pudo haberlo generado).

El MAC da **integridad** y **autenticación de origen** entre las partes que comparten la clave, pero **no da no-repudio**.

El **no-repudio** requiere **firma digital** (criptografía **asimétrica**: clave privada que solo posee el firmante).

8. Memory hooks

Destacado

Mnémonicos para el examen:

- **Mac-Forge:** “*Falsificar = inventar un tag nuevo para un mensaje nuevo*”. El oráculo modela los pares (m, t) que el atacante observa en tránsito; la restricción $m^* \notin Q$ cierra el ataque trivial.
- **CBC-MAC IV=0:** “*No ciframos, no necesitamos IV aleatorio*”. El encadenamiento hace que t_ℓ dependa de todos los bloques; no importa el valor inicial siempre que sea fijo y conocido.
- **CBC-MAC = solo longitud fija:** “*Longitud variable $\Rightarrow$ ataque de extensión*”. El truco: prefijo + XOR del estado intermedio.
- **Las 3 construcciones:** E&M (en paralelo) = NO, MtE = Puede, EtM = Sí. Regla: “*el MAC siempre sobre el cifrado, con claves distintas*”.
- **No-repudio:** “*Simétrico $\Rightarrow$ ambos pueden forjar $\Rightarrow$ ninguno puede repudiar al otro ante terceros*”. Solo la firma digital (asimétrica) lo resuelve.

9. Ejercicios tipo examen con respuesta

9.1. V/F: hash, MAC y cifrado (parciales 2016/2018/2023/2025)

[Examen]

1. “Un hash y un MAC son lo mismo” $\rightarrow$ **FALSO**. El hash no tiene clave; el MAC sí. El hash no da autenticación; el MAC sí.
2. “Un MAC garantiza confidencialidad” $\rightarrow$ **FALSO**. El MAC etiqueta el *plano*: el mensaje viaja en claro. No hay cifrado.
3. “Encrypt-and-MAC con $k_1 = k_2$ es seguro” $\rightarrow$ **FALSO** (parciales 2016, 2018, 2023). Incluso con claves distintas Encrypt-and-MAC no es seguro en general; además, usar la misma clave para cifrado y MAC introduce vulnerabilidades adicionales.
4. “MAC-then-Encrypt siempre es seguro” $\rightarrow$ **FALSO**. No hay prueba genérica; solo pruebas para combinaciones concretas.
5. “Encrypt-then-MAC siempre es seguro si el cifrado es CPA y el MAC es EUF-CMA” $\rightarrow$ **VERDADERO**. Hay prueba formal genérica.

9.2. Encrypt-and-MAC inseguro (parciales 2016/2018/2023)

[Examen]

¿Por qué Encrypt-and-MAC es inseguro?

La definición de MAC cubre infalsificabilidad (integridad), no confidencialidad. El tag $t = \text{Mac}_k(m)$ es una función determinista del plano m (muchos MACs son deterministas). Publicar t junto al cifrado puede filtrar información sobre m : por ejemplo, si el adversario conoce algunos candidatos a m , puede calcular el tag de cada uno y compararlo con t para identificar el plano.

Contraejemplo concreto: sea $\text{Mac}_k(m) = F_k(m)$ (PRF aplicada al plano). El tag $t = F_k(m)$ es determinista en m ; el adversario puede distinguir entre m_0 y m_1 comparando tags. La construcción falla CPA para el sistema combinado aunque el cifrado sea CPA-Secure.

9.3. HMAC para CVV (parcial 2017)

[Examen]

Se quiere verificar un CVV sin almacenarlo. ¿Qué construcción usar?

Almacenar $\text{HMAC}_k(\text{CVV})$ en lugar del CVV en claro. Al verificar: recalcular $\text{HMAC}_k(\text{CVV recibido})$ y comparar. Si coincide, el CVV es correcto.

Propiedades:

- Quien no tiene k no puede generar un HMAC válido (infalsificabilidad).
- Si la base de datos se filtra, los CVV no quedan expuestos (resistencia a preimagen del hash subyacente más la clave del HMAC).
- **No da no-repudio** (clave simétrica compartida entre el sistema emisor y el verificador).

9.4. Construcción $c = E_{k_1}(m \parallel H(k_2 \parallel m))$ (parcial 2025-1)

[Examen]

¿Qué hace el receptor? ¿Qué garantiza la construcción?

1. Descifrar: $m \parallel h' \leftarrow \text{Dec}_{k_1}(c)$.
2. Recomputar: $h = H(k_2 \parallel m)$.
3. Comparar: si $h = h'$, el mensaje es íntegro y autenticado; si no, rechazar.

Garantiza:

- **Confidencialidad:** E_{k_1} cifra todo (mensaje y hash).
- **Integridad y autenticación:** $H(k_2 \parallel m)$ es esencialmente un “HMAC casero” (MAC de clave prefijo); solo quien tiene k_2 puede generarlo.

NO garantiza: no-repudio (clave simétrica compartida k_2 ; el receptor también puede construir $H(k_2 \parallel m)$).

¿Es Encrypt-then-MAC? Casi: el MAC es “MAC-then-Encrypt” porque se calcula $H(k_2 \parallel m)$ sobre el plano y luego se cifra todo. Comparte las propiedades de seguridad de MAC-then-Encrypt (prueba específica, no genérica), no la prueba genérica de Encrypt-then-MAC puro.